



Exercices — Jour 3

React Native · Modules 5, 6 & 7 : Données, API natives & Avancé

Progression : les exercices 1-3 couvrent formulaires et données, 4-5 les API natives, 6 les tests, 7 la publication et le projet final.

Exercice 1 — Formulaire d'inscription avec React Hook Form (30 min)

Objectif : créer un formulaire complet, validé et accessible.

Consigne

Créez `app/inscription.tsx` — un formulaire d'inscription en 3 étapes.

```
npm install react-hook-form @hookform/resolvers zod
```

Schéma de validation Zod :

```
import { z } from 'zod';

const etape1Schema = z.object({
  prenom: z.string().min(2, 'Minimum 2 caractères').max(50),
  nom: z.string().min(2).max(50),
  email: z.string().email('Adresse email invalide'),
});

const etape2Schema = z.object({
  telephone: z.string().regex(/^\+(\+33|0)[1-9](\d{8})$/, 'Format invalide'),
  dateNaissance: z.string().regex(/^\d{2}\d{2}\d{4}$/, 'Format JJ/MM/AAAA'),
  codePostal: z.string().length(5, 'Code postal à 5 chiffres'),
});

const etape3Schema = z.object({
  motDePasse: z.string()
    .min(8, 'Minimum 8 caractères')
    .regex(/[A-Z]/, 'Au moins une majuscule')
    .regex(/[0-9]/, 'Au moins un chiffre'),
  confirmation: z.string(),
  cgu: z.literal(true, { errorMap: () => ({ message: 'Obligatoire' }) }),
}).refine(d => d.motDePasse === d.confirmation, {
  message: 'Les mots de passe ne correspondent pas',
  path: ['confirmation'],
});
```

Fonctionnalités :

1. Indicateur de progression (Étape 1/3, 2/3, 3/3)
2. Validation à chaque étape avant de passer à la suivante
3. Messages d'erreur affichés sous chaque champ en rouge
4. Bouton "Précédent" pour revenir (les données sont conservées)
5. `ActivityIndicator` pendant la soumission finale (simuler avec `setTimeout`)
6. `Snackbar` de succès ou d'erreur après soumission

Accessibilité

- `accessibilityLabel` sur chaque `TextInput`
- `returnKeyType="next"` et focus automatique sur le champ suivant
- Le formulaire défile si le clavier cache des champs (`KeyboardAvoidingView`)

✓ Critères de validation

- Aucun champ ne peut être ignoré (validation stricte)
- Les erreurs disparaissent dès que le champ est corrigé (mode: 'onChange')
- Le formulaire est utilisable au clavier sans toucher l'écran

● Exercice 2 — Couche API : Service et cache (25 min)

Objectif : structurer les appels réseau avec Axios et TanStack Query.

Consigne

Créez une couche de service API complète pour l'API JSONPlaceholder.

Architecture à créer :

```
lib/  
├─ api.ts           → Instance Axios configurée  
├─ services/  
│   ├─ users.ts     → CRUD utilisateurs  
│   └─ posts.ts     → CRUD articles  
└─ hooks/  
    ├─ useUsers.ts   → Hooks TanStack Query pour users  
    └─ usePosts.ts   → Hooks TanStack Query pour posts
```

lib/api.ts à compléter :

```
import axios, { AxiosError } from 'axios';

const api = axios.create({
  baseURL: 'https://jsonplaceholder.typicode.com',
  timeout: 10_000,
});

// Intercepteur : logger les requêtes en développement
// Intercepteur : transformer les erreurs en messages lisibles
// Intercepteur : retry automatique si erreur réseau (max 3 fois)

export default api;
```

Hooks à créer :

```
// lib/hooks/usePosts.ts
export function usePosts() // Liste avec cache 5min
export function usePost(id: number) // Détail
export function useCreatePost() // Mutation + invalidation cache
export function useDeletePost() // Mutation + optimistic update
```

Interface à développer :

1. Liste d'articles avec auteur (jointure users + posts)
2. Formulaire de création d'article
3. Suppression avec confirmation (optimistic update : l'article disparaît immédiatement)
4. Gestion offline : afficher un bandeau si pas de connexion

✓ Critères de validation

- Les requêtes ne se déclenchent pas inutilement (vérifier dans les DevTools Network)
- L'optimistic update fonctionne : l'UI s'update avant la réponse serveur
- Les erreurs réseau affichent un message compréhensible (pas un objet brut)

● Exercice 3 — Stockage local : Journal hors-ligne (30 min)

Objectif : implémenter une app fonctionnelle 100% hors-ligne avec synchronisation.

Consigne

Créez `app/journal.tsx` — un journal personnel avec sync serveur.

```
npm install @react-native-async-storage/async-storage
# ou pour les performances :
npm install react-native-mmkv
```

Structure d'une entrée :

```
interface EntreeJournal {
  id: string;
  titre: string;
  contenu: string;
  humeur: '😊' | '😐' | '😬' | '😭' | '🥳';
  creeLe: number;
  modifieLe: number;
  synchronisee: boolean; // true si envoyée au serveur
}
```

Fonctionnalités :

1. Créer / modifier / supprimer des entrées
2. Sauvegarder immédiatement en local (MMKV ou AsyncStorage)
3. Indicateur de connexion (NetInfo) dans le header
4. Si connecté : synchroniser automatiquement les entrées `synchronisee: false`
5. Mode lecture offline : toutes les entrées sont accessibles sans réseau
6. Recherche plein texte dans les entrées (titre + contenu)

Schéma de stockage :

```
// Clés de stockage
const CLES = {
  ENTREES: 'journal:entries',
  DERNIERE_SYNC: 'journal:lastSync',
} as const;
```

Bonus

- Export des entrées en fichier texte (expo-sharing)

- Chiffrement des données sensibles (expo-secure-store pour la clé)
- Pagination des entrées (ne charger que les 20 dernières)

✓ Critères de validation

- Les données persistent après fermeture complète de l'app
- La synchronisation ne se déclenche que si `isConnected === true`
- La recherche fonctionne sans requête réseau

● Exercice 4 — API natives : App "Météo & Position" (35 min)

Objectif : combiner géolocalisation, notifications et capteurs.

Consigne

Créez une application météo qui utilise la position GPS.

```
npx expo install expo-location expo-notifications expo-sensors
```

API météo gratuite (100 requêtes/jour sans clé) :

```
https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m,wir
```

Fonctionnalités :

1. **Géolocalisation** : demander la permission, obtenir lat/lon
2. **Météo actuelle** : température, vitesse du vent, condition
3. **Prévisions 7 jours** : min/max par jour (SectionList)
4. **Notification programmée** : rappel météo chaque matin à 8h00
5. **Altimètre bonus** : utiliser le baromètre (`Barometer`) pour estimer l'altitude
6. **Partage** : bouton "Partager la météo" (`Share.share`)

```
// Mapper le code météo WMO vers une description
const CODES_METEO: Record<number, { label: string; emoji: string }> = {
  0: { label: 'Ciel dégagé', emoji: '☀️' },
  1: { label: 'Peu nuageux', emoji: '☁️' },
  2: { label: 'Partiellement nuageux', emoji: '⛅️' },
  3: { label: 'Nuageux', emoji: '☁️' },
  61: { label: 'Pluie légère', emoji: '🌧️' },
  63: { label: 'Pluie modérée', emoji: '🌧️' },
  71: { label: 'Neige légère', emoji: '❄️' },
  95: { label: 'Orage', emoji: '⚡️' },
};
```

Gestion des permissions

```
// Pattern à suivre pour toutes les permissions
async function demanderPermission() {
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status === 'denied') {
    // Expliquer pourquoi et proposer d'ouvrir les réglages
    Alert.alert(
      'Permission requise',
      'La localisation est nécessaire pour afficher la météo locale.',
      [
        { text: 'Annuler', style: 'cancel' },
        { text: 'Paramètres', onPress: () => Linking.openSettings() },
      ]
    );
  }
}
```

✅ Critères de validation

- Les permissions sont demandées proprement avec explication
- Si permission refusée, l'app propose une alternative (saisir la ville manuellement)
- La notification est annulée si l'app est désinstallée (cleanup)

● Exercice 5 — Caméra & traitement d'image (30 min)

Objectif : implémenter un scanner de QR code et un traitement d'image basique.

Consigne

Créez `app/scanner.tsx` — un scanner de QR code avec historique.

```
npx expo install expo-camera expo-image-picker expo-image-manipulator
```

Fonctionnalités :

1. **Scanner QR** : détecter et décoder un QR code en temps réel
2. **Historique** : sauvegarder les 10 derniers QR scannés (MMKV)
3. **Actions selon le type** :
 - URL → ouvrir dans le navigateur (Linking)
 - Email → ouvrir le client mail
 - Téléphone → proposer d'appeler
 - Texte brut → copier dans le presse-papiers
4. **Photo de profil** : sélectionner depuis la galerie, recadrer en 1:1, compresser à 80%

```
// Traitement d'image avec expo-image-manipulator
import * as ImageManipulator from 'expo-image-manipulator';

const traiter = async (uri: string) => {
  const result = await ImageManipulator.manipulateAsync(
    uri,
    [
      { resize: { width: 400 } },
      { crop: { originX: 0, originY: 0, width: 400, height: 400 } },
    ],
    { compress: 0.8, format: ImageManipulator.SaveFormat.JPEG }
  );
  return result.uri;
};
```


✓ Critères de validation

- Le scanner s'arrête après détection (pas de scans en boucle)
- L'historique est limité à 10 entrées (FIFO)
- L'image de profil est bien recadrée en carré

● Exercice 6 — Tests complets (35 min)

Objectif : écrire une suite de tests unitaires et d'intégration.

Consigne

Écrivez les tests pour les composants suivants.

6a — Test d'un reducer Redux :

```
// __tests__/tasksSlice.test.ts
import tasksReducer, { ajouter, basculerFaite, supprimer }
  from '../store/tasksSlice';

describe('tasksReducer', () => {
  const etatInitial = [];

  it('ajoute une tâche', () => {
    // Écrire le test
  });

  it('bascule l'état "faite" d'une tâche', () => {
    // Écrire le test – créer une tâche puis la basculer
  });

  it('supprime une tâche par id', () => {
    // Écrire le test
  });

  it('ne modifie pas les autres tâches lors d'une suppression', () => {
    // Écrire le test avec 3 tâches, en supprimer une, vérifier les 2 restantes
  });
});
```

6b — Test d'un composant :

```
// __tests__/CartePresentation.test.tsx
import { render, fireEvent } from '@testing-library/react-native';
import CartePresentation from '../components/CartePresentation';

const propsDefault = {
  prenom: 'Alice',
  nom: 'Dupont',
  role: 'Développeur',
  ville: 'Paris',
  photoUrl: 'https://example.com/photo.jpg',
  onContact: jest.fn(),
};

describe('CartePresentation', () => {
  it('affiche le nom complet', () => { /* ... */ });
  it('affiche le rôle et la ville', () => { /* ... */ });
  it('appelle onContact au press du bouton', () => { /* ... */ });
  it('affiche une image avec le bon accessibilityLabel', () => { /* ... */ });
});
```

6c — Test d'un hook :

```
// __tests__/useFetch.test.ts
import { renderHook, waitFor } from '@testing-library/react-native';
import { useFetch } from '../lib/hooks/useFetch';

// Mocker fetch
global.fetch = jest.fn();

describe('useFetch', () => {
  beforeEach(() => jest.clearAllMocks());

  it('retourne loading=true pendant le chargement', () => { /* ... */ });
  it('retourne les données après succès', async () => { /* ... */ });
  it('retourne error si la requête échoue', async () => { /* ... */ });
  it('ne met pas à jour le state si le composant est démonté', async () => { /* ... */ });
});
```

✓ Critères de validation

- Couverture de code $\geq 70\%$ (`npx jest --coverage`)
- Aucun test ne dépend d'un autre (ordre indépendant)
- Les mocks sont correctement nettoyés entre les tests (`afterEach`)

● Exercice 7 — Projet final : Application de production (90 min)

Objectif : déployer une application complète prête pour les stores.

Consigne — Application "Recettes"

Construisez et préparez au déploiement une application de recettes de cuisine.

API gratuite :

<https://www.themealdb.com/api/json/v1/1/search.php?s=chicken>

<https://www.themealdb.com/api/json/v1/1/lookup.php?i={id}>

<https://www.themealdb.com/api/json/v1/1/categories.php>

Fonctionnalités requises :

Fonctionnalité	Module
Recherche recettes (debounce 300ms)	Module 5
Favoris persistés MMKV	Module 5
Fiche recette avec vidéo YouTube	Module 6
Notification "Heure de cuisiner !" programmable	Module 6
Partage de recette	Module 6
Tests unitaires (≥ 5 tests)	Module 7
Build EAS configuré	Module 7
Mode sombre complet	Transversal

Checklist de préparation au déploiement :

1. Qualité du code

<code>npx jest --coverage</code>	<code># ≥ 70%</code>
<code>npx eslint . --ext .tsx,.ts</code>	<code># 0 erreur</code>
<code>npx tsc --noEmit</code>	<code># 0 erreur TypeScript</code>

2. Configuration app.json

#	- version: "1.0.0"
#	- bundleIdentifier (iOS)
#	- package (Android)
#	- icon et splash screen personnalisés

3. Variables d'environnement

<code>npx eas secret:create</code>	<code>--name API_URL</code>	<code>--value "..."</code>
------------------------------------	-----------------------------	----------------------------

4. Build

<code>npx eas build</code>	<code>--platform android</code>	<code>--profile preview</code>
----------------------------	---------------------------------	--------------------------------

Livrables attendus :

1. Code source dans un repo Git propre (commits atomiques, messages clairs)
2. README.md avec instructions d'installation et de lancement
3. Lien vers le build EAS (ou APK signé)
4. Rapport de tests (`jest --coverage`)

Critères de validation

- L'application ne crash pas sur les scénarios courants (liste vide, pas de réseau)
- Le mode sombre est cohérent dans toute l'application
- Les permissions natives sont demandées proprement avec justification
- Le build de production se compile sans erreur